Virtualización del SO

Sistemes Operatius II

Oliver Díaz

Grau d'Enginyeria Informàtica

¿Como afrontáis la entrevista es esta tarde/mañana?



Noticias

- Enunciado de la **P2 (Dockers)** disponible al CV
- La próxima semana tendremos **test** de esta unidad
- Hoy/mañana **entrevista de la P1**









Objetivos de la sesión

- Introducción a los contenedores
- Identificar los entornos dónde podemos utilizar contenedores
- Familiarización con la tecnología Docker
- Aprender/repasar commandos básicos de Docker
- Dar soporte a la P2 del laboratorio

Durante la sesión iremos trabajando preguntas sobre contenedores para **buscar/reflexionar** en grupo





¿Qué vimos la semana pasada?

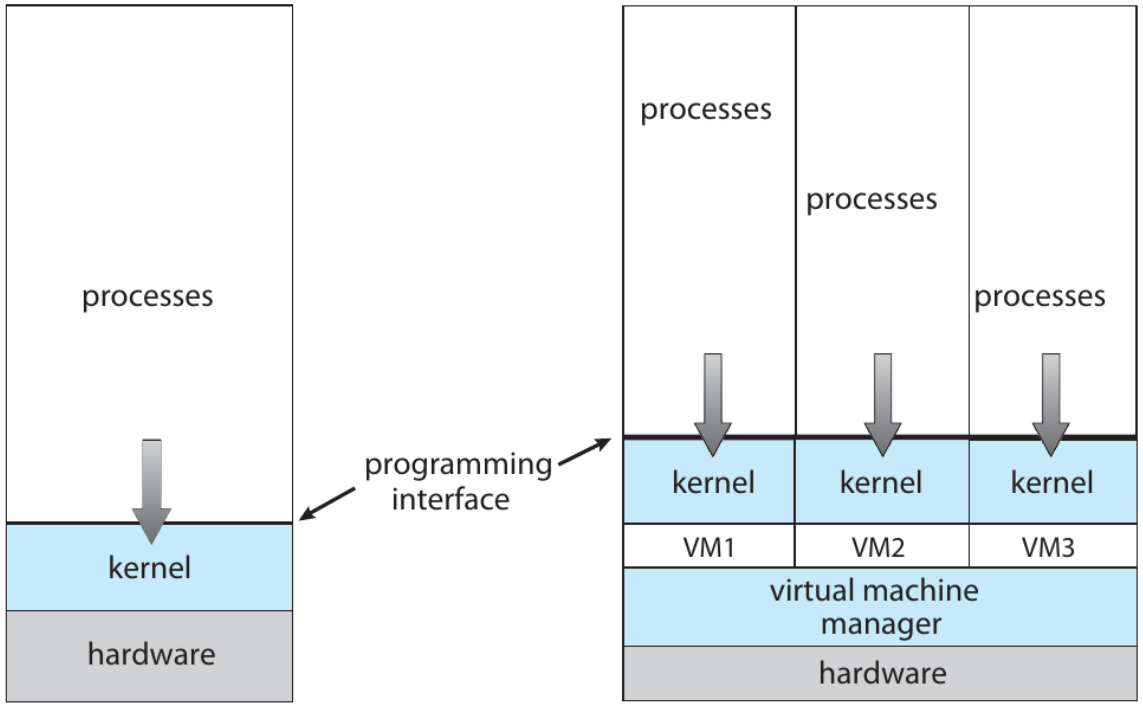


"I forgot to make a back-up copy of my brain, so everything I learned last ~~semester~~ week was lost."



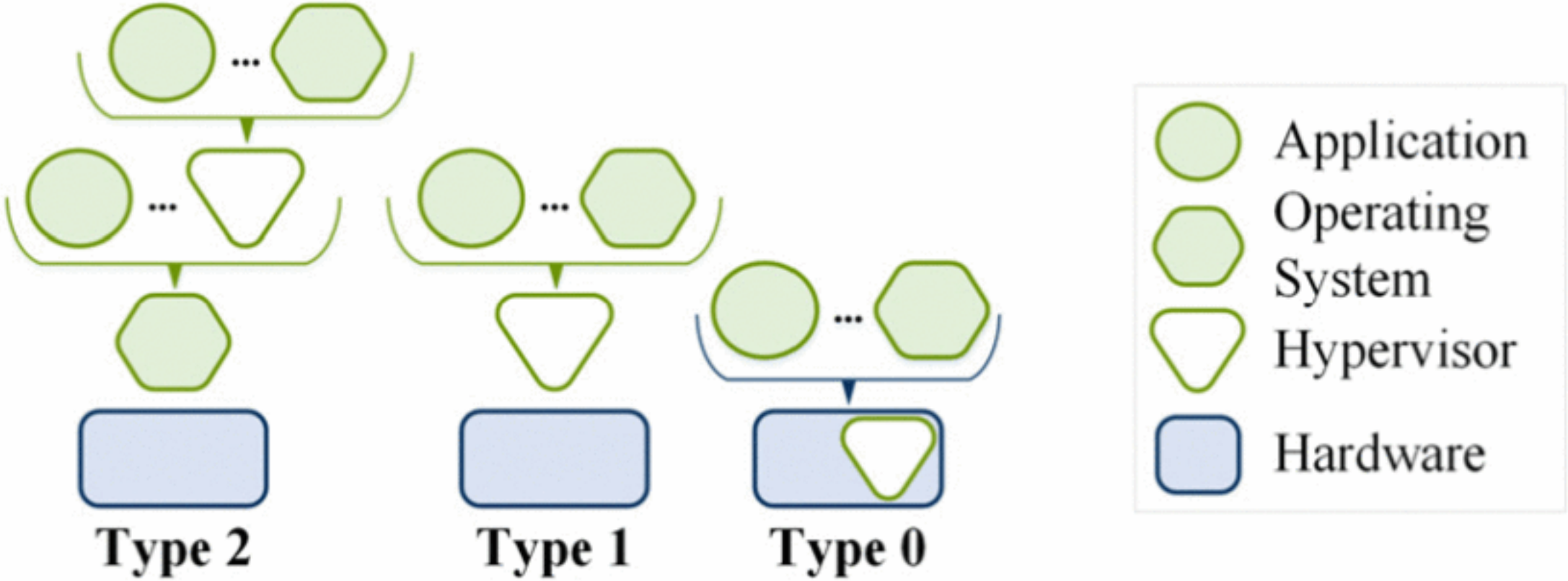
¿Qué vimos la semana pasada?

- Las MVs son una **abstracción de la capa del hardware**.
- *Hypervisors* (VMM) permiten ejecutar 1 (o más) MVs en una máquina física.
- Cada MV es una **copia completa del SO**, aplicaciones, binarios y librerias.



Silberschatz, A., Peterson, J. L., & Galvin, P. B. (2018). *Operating system concepts*. Wiley

¿Qué vimos la semana pasada?

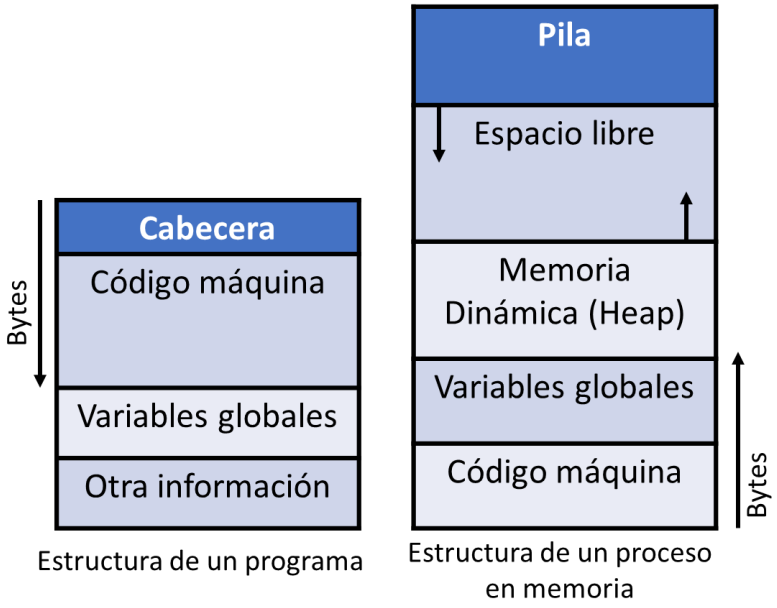


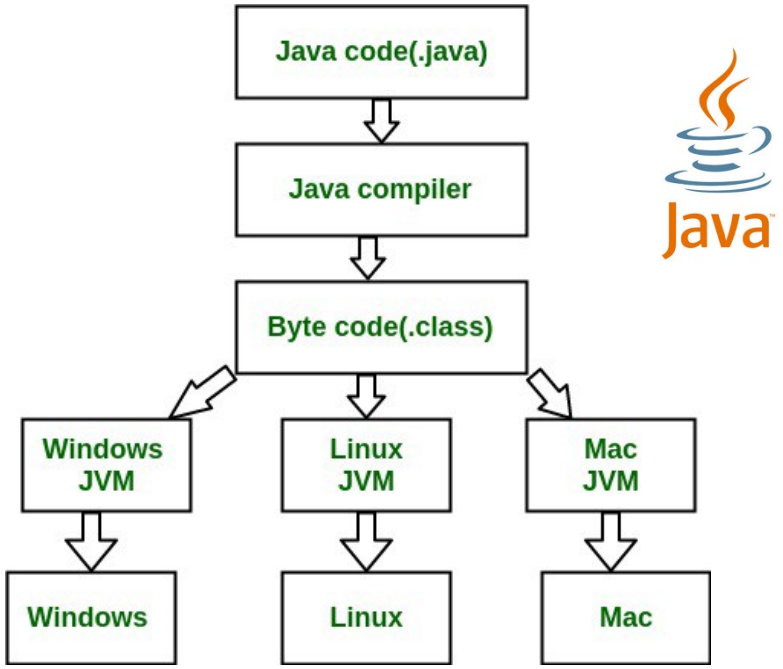
Janßen, Benedikt, et al. "Towards a type 0 hypervisor for dynamic reconfigurable systems." 2017 International Conference on ReConFigurable Computing and FPGAs (ReConFig). IEEE, 2017.

¿Qué vimos la semana pasada?

- La **virtualización de procesos** nos permite ejecutar un proceso en un entorno independiente. La MV es la encargada de lidiar con el SO.

Proceso: Instancia de un programa

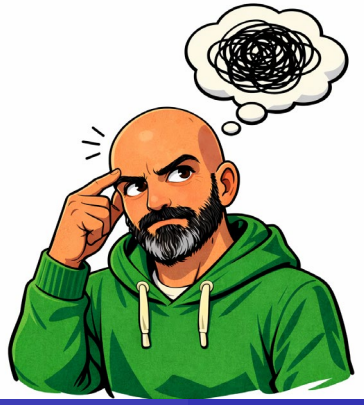




¿Qué vimos la semana pasada?

¿De verdad que nos interesa virtualizar siempre

todos los recursos de una máquina física?



¿Qué vimos la semana pasada?

- La **virtualización** tiene por objeto proporcionar un método para **segregar aplicaciones**, **administrar** su rendimiento y uso de recursos, y disponer de una **forma fácil** de iniciarlos, detenerlos, moverlos y administrarlos.
- En estos casos, tal vez no es necesario una virtualización total de los recursos y SO.
- Por ese motivo, en determinadas aplicaciones resulta más adecuado utilizar la **virtualización a nivel de SO** (*OS-level virtualization*)

Virtualización a nivel de SO

*"OS-level virtualization is an **OS paradigm** in which the **kernel** allows the existence of **multiple isolated user space instances**, called **containers** (LXC, Solaris containers, Docker, Podman), **zones** (Solaris containers), **virtual private servers** (OpenVZ), **partitions**, **virtual environments** (VEs), **virtual kernels** (DragonFly BSD), or **jails** (FreeBSD jail or chroot jail).*

*Such instances may **look like real computers** from the point of view of programs running in them"*

https://en.wikipedia.org/wiki/OS-level_virtualization

Analicemos este caso

- **Héctor** tiene en su ordenador instalado **Python 2.7**, y está programando una funcionalidad específica de una aplicación con algo que **solo está disponible en esa versión de Python.**
- **María** tiene instalado en su máquina **Python 3.6**, porque está en otro proyecto trabajando sobre otro código, pero **Héctor quiere que María ejecute el código de su aplicación en su máquina**

P1. ¿Qué posibles escenarios podrían darse en esta situación?



María instala Python 2.7

María instala MV + Python 2.7

Héctor crea un **contenedor** con la aplicación y todas las librerías (Python 2.7) y recursos necesarios

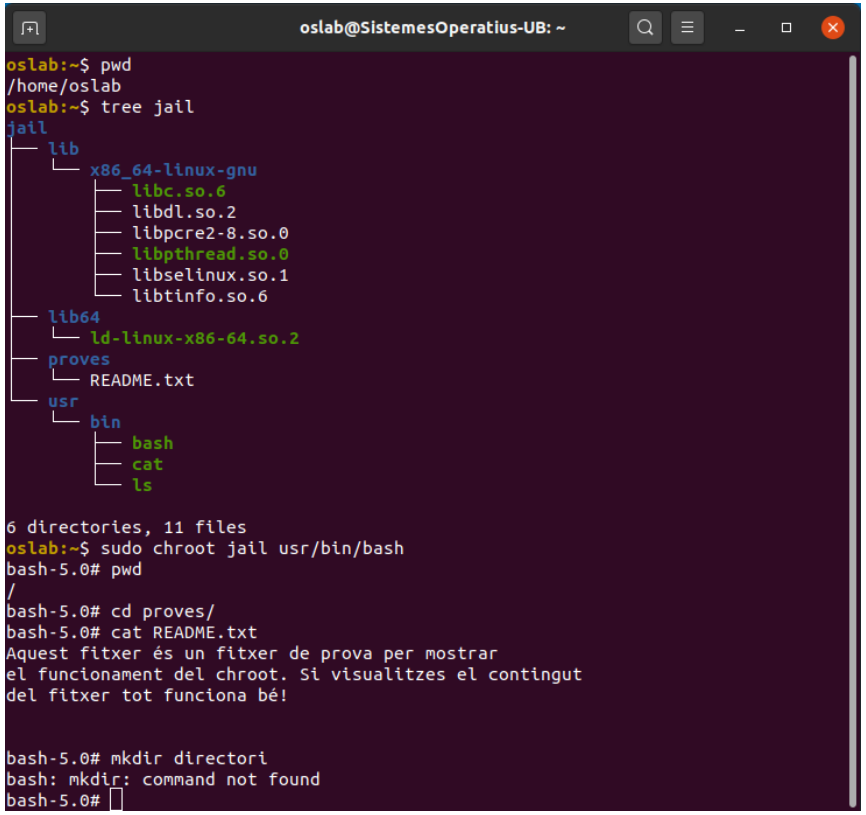
Contenedores

- Los **contenedores** corresponden a la **abstracción de la capa de aplicaciones** que contiene un código y sus dependencias. **MV → abstracción de la capa del hardware**.
- Permite crear **espacios de usuarios aislados** para ejecutar aplicaciones de forma rápida y fiable, independientemente del entorno informático (SO).
- Los procesos ejecutados dentro del contenedor solo ven los **recursos asignados**.
- **Múltiples contenedores** pueden ejecutarse en una máquina (o en varias).



Contenedores

- El **`chroot jail`** (1979) es un comando de **Unix** que permite modificar el entorno de ejecución de forma que, al ejecutar un proceso, este proceso (y sus correspondientes hijos) sólo pueden "ver" los archivos que hay bajo un directorio raíz especificado (e.g. /jail)



Contenedores

P2 (a) Enumera las ventajas que aporta programar con contenedores.
(b) ¿Qué diferencias existen entre las MV y los contenedores?



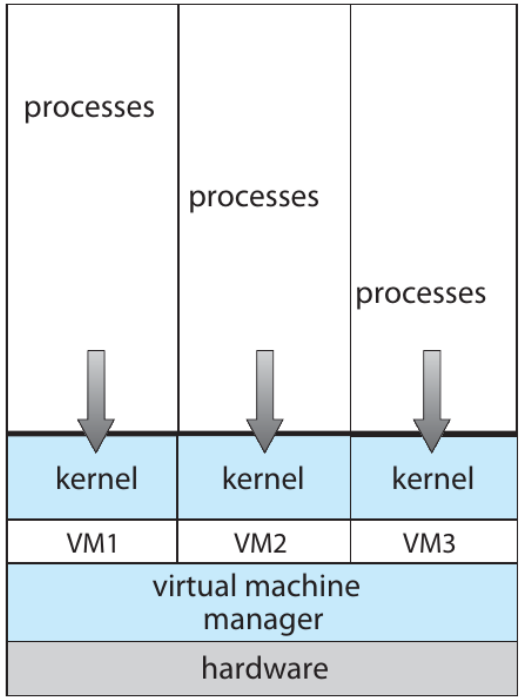
- Los procesos realizan **llamadas al sistema** directamente al SO por lo que no es necesario un sistema de virtualización (*hypervisor*) como en una MV.
- Permiten ejecutar apps en contenedores diferentes (seguridad)
- En sistemas de alojamiento, cada usuario tiene asignado un contenedor con **recursos limitados**.
- Tecnologías como **Docker**, permiten la fácil distribución del contenedor entero a sus clientes, facilitando su instalación y mantenimiento.

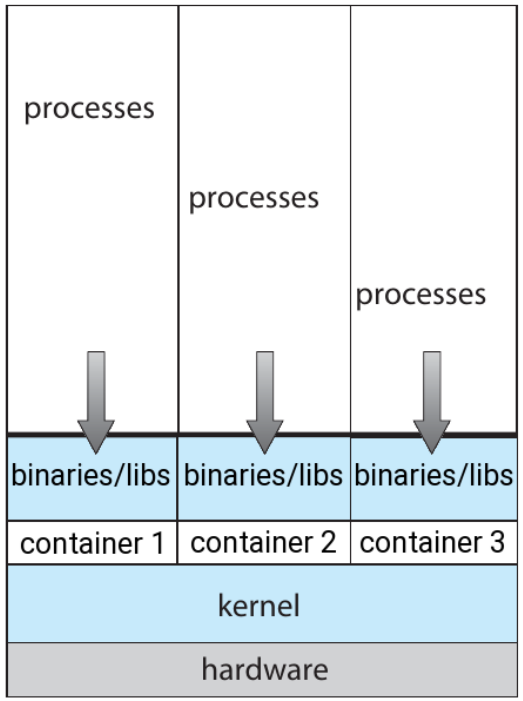
Contenedores vs MVs

P2 (a) Enumera las ventajas que aporta programar con contenedores.
(b) ¿Qué diferencias existen entre las MV y los contenedores?



- Las **MVs** virtualizan el hardware de la máquina. Cada MV puede ejecutar un SO diferente. Lentos de inicializar. Tamaño considerable (GBs)
- Los **contenedores** virtualizan el SO. Son más portátiles y eficientes. Cada contenedor se ejecuta sobre un único SO. Rápidos y ligeros (MBs)





Silberschatz, A., Peterson, J. L., & Galvin, P. B. (2018). *Operating system concepts*. Wiley

Contenedores

Los contenedores han evolucionado durante los últimos años de tal forma que **controlan el espacio de usuario** en el que se ejecutan los programas.

P3. ¿Qué tipo de recursos pueden ser controlados por un contenedor?



- Espacio de directorio/fichero al cual pueden acceder
- Número máximo de CPUs que pueden utilizar.
- Memoria RAM
- Número máximo de procesos que se ejecutan dentro del contenedor
- …

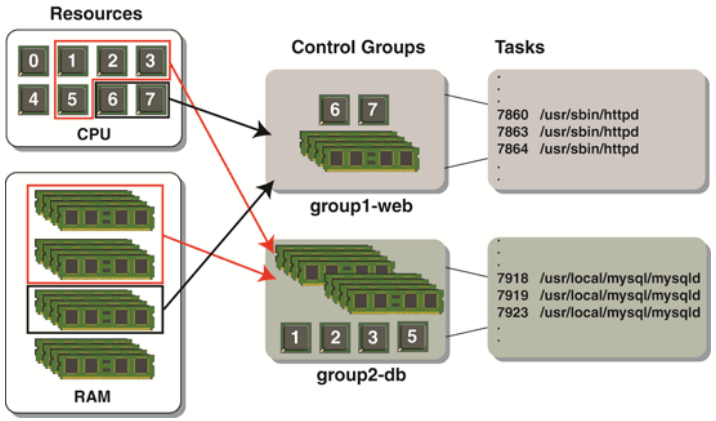
Virtualización a nivel de SO

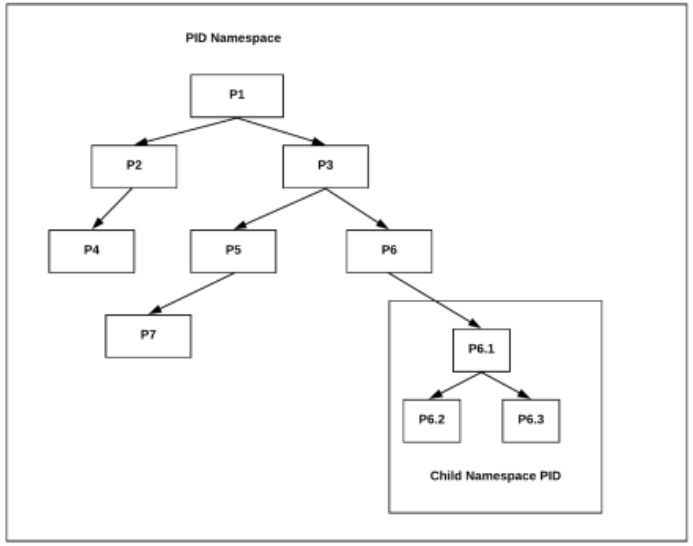
Los **contenedores** están gestionados internamente por el SO. En el caso de Linux…

P4. ¿Qué tecnologías son utilizadas actualmente para gestionar el acceso a estos recursos?



- **Control groups (cgroups):** tecnología del SO que permite limitar los recursos de un conjunto de procesos.
- **Namespaces**: tecnología que permite aislar los contenedores entre sí. Cada proceso dentro de un contenedor tendrá su propio PID. Cada contenedor tendrá su propio *cgroup*.





Virtualización a nivel de SO

- **Docker Desktop** es la tecnología que permite ejecutar Docker Windows y Mac (https://docs.docker.com/desktop/). Docker, sin embargo, fue desarrollado gracias a los servicios que ofrece el *kernel* Linux, en particular **cgroups** y **namespaces**.

P5. ¿Cómo es posible entonces ofrecer servicios Docker Windows?

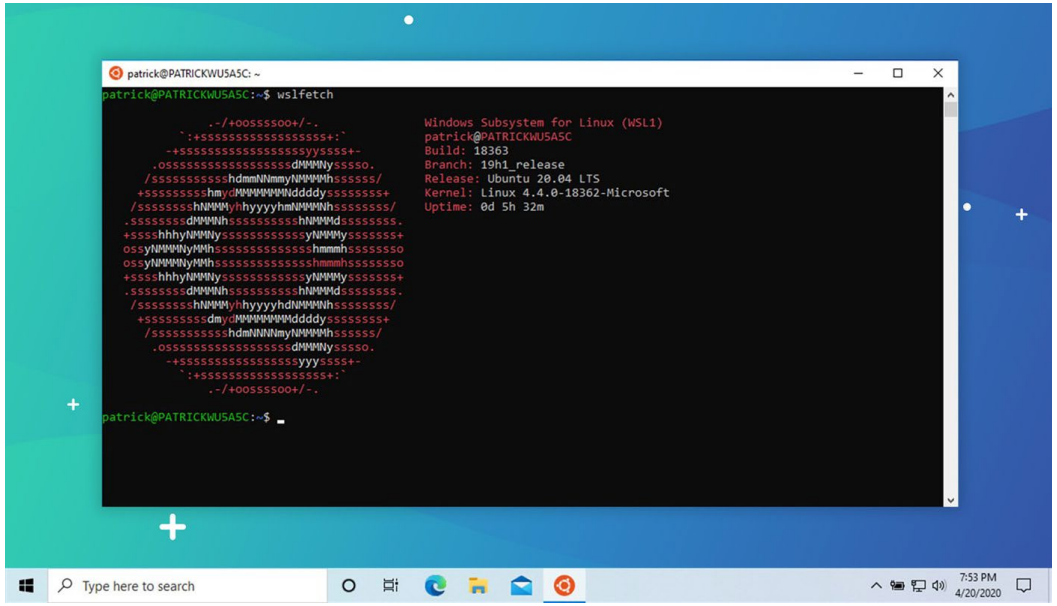


- **Windows Hyper-V virtualization**
- Habilidad de funcionar nativamente (sin emulación) en Linux mediante "**Windows Subsystem for Linux" (WSL) 2**

WSL 2

- *"**WSL 2** introduces a significant architectural change as it is **a full Linux kernel built by Microsoft**, allowing Linux **containers to run natively without emulation**. With Docker Desktop running on WSL 2, users can leverage **Linux workspaces** and avoid having to maintain both Linux and Windows build scripts. In addition, WSL 2 provides improvements to file system sharing, boot time, and allows access to some cool new features for Docker Desktop users."*

https://docs.docker.com/docker-for-windows/wsl/



Virtualización a nivel de SO

En la actualidad, existen un gran número de gestores de contenedores que están disponibles para que los programadores puedan crear y ejecutar sus programas.

P6. (a) Enumera 3 gestores de contenedores. (b) Revisa la web oficial de uno de ellos.



docker

https://www.docker.com/

rkt

https://coreos.com/rkt/

cri-o

https://cri-o.io/

LXC

https://linuxcontainers.org/

Virtualización a nivel de SO

https://en.wikipedia.org/wiki/OS-level_virtualization

Mechanism	Operating system	License	Actively developed since or between	Features									
				File system isolation	Copy on Write	Disk quotas	I/O rate limiting	Memory limits	CPU quotas	Network isolation	Nested virtualization	Partition checkpointing and live migration	Root privilege isolation
chroot	Most UNIX-like operating systems	Varies by operating system	1982	Partial[a]	No	No	No	No	No	No	Yes	No	No
Docker	Linux,[9] FreeBSD,[10] Windows x64 (Pro, Enterprise and Education)[11] macOS[12]	Apache License 2.0	2013	Yes	Yes	Not directly	Yes (since 1.10)	Yes	Yes	Yes	Yes	Only in Experimental Mode with CRIU [1]	Yes (since 1.10)
FreeBSD jail	FreeBSD, DragonFly BSD	BSD License	2000[28]	Yes	Yes (ZFS)	Yes[s]	Yes	Yes[29]	Yes	Yes[30]	Yes	Partial[31][32]	Yes[33]
iCore Virtual Accounts	Windows XP	Freeware	2008	Yes	No	Yes	No	No	No	No	?	No	?
Linux-VServer (security context)	Linux, Windows Server 2016	GNU GPLv2	2001	Yes	Yes	Yes	Yes[b]	Yes	Yes	Partial[c]	?	No	Partial[d]
lmctfy	Linux	Apache License 2.0	2013–2015	Yes	Yes	Yes	Yes[b]	Yes	Yes	Partial[c]	?	No	Partial[d]
LXC	Linux	GNU GPLv2	2008	Yes[14]	Yes	Partial[e]	Partial[f]	Yes	Yes	Yes	Yes	Yes	Yes[14]
OpenVZ	Linux	GNU GPLv2	2005	Yes	Yes[18]	Yes	Yes[g]	Yes	Yes	Yes[h]	Partial[i]	Yes	Yes[j]
rkt	Linux	Apache License 2.0	2014[42]–2018	Yes	Yes	Yes	Yes	Yes	Yes	Yes	?	?	Yes
Sandboxie	Windows	GNU GPLv3	2004	Yes	Yes	Partial	No	No	No	Partial	No	No	Yes
Singularity	Linux	BSD Licence	2015[15]	Yes[16]	Yes	Yes	No	No	No	No	No	No	Yes[17]
Solaris Containers (Zones)	illumos (OpenSolaris), Solaris	CDDL, Proprietary	2004	Yes	Yes (ZFS)	Yes	Partial[m]	Yes	Yes	Yes[n][25][26]	Partial[o]	Partial[p][q]	Yes[r]
sysjail	OpenBSD, NetBSD	BSD License	2006–2009	Yes	No	No	No	No	No	Yes	No	No	?
systemd-nspawn	Linux	GNU LGPLv2.1+	2010	Yes	Yes	Yes[40][41]	Yes[40][41]	Yes[40][41]	Yes[40][41]	Yes	?	?	Yes
Turbo	Windows	Freemium	2012	Yes	No	No	No	No	No	Yes	No	No	Yes
Virtuozzo	Linux, Windows	Trialware	2000[22]	Yes	Yes	Yes	Yes[k]	Yes	Yes	Yes[h]	Partial[l]	Yes	Yes
vkernel	DragonFly BSD	BSD Licence	2006[34]	Yes[35]	Yes[35]	—	?	Yes[36]	Yes[36]	Yes[37]	?	?	Yes
WPARs	AIX	Commercial proprietary software	2007	Yes	No	Yes	Yes	Yes	Yes	Yes[t]	No	Yes[39]	?

Docker



Docker

La tecnología (*open source*) de contenedores Docker apareció en 2013. Internamente utiliza la tecnología de los **cgroups** y **namespaces**. Docker añade múltiples características para manipular y ejecutar contenedores, de forma equivalente:

- Docker asegura **portabilidad** de sus contenedores entra máquinas que lo utilizan. Eso permite que un contenedor contenga todas las librerías necesarias para ejecutar una aplicación.

- Docker permite crear contenedores de forma sencilla. Incluye funcionalidades de gestión de versiones similar a Git.



Docker: definiciones

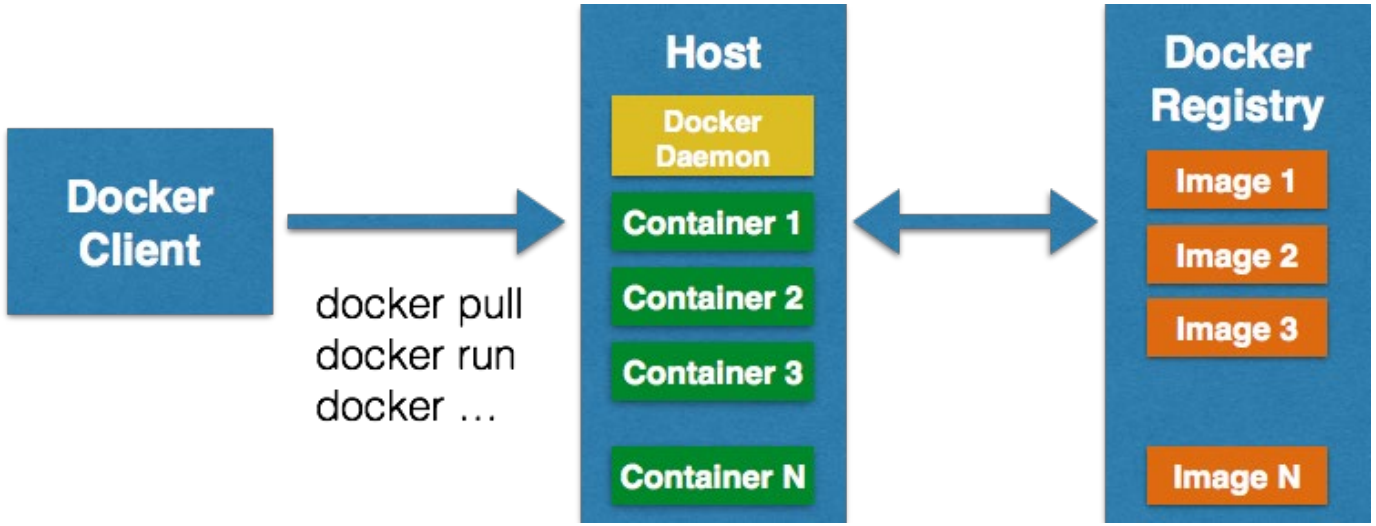


P7. Consulta la página oficial de Docker y describe los siguientes términos que se utilizan en esta tecnología:

(a) Docker Registry / DockerHub
(b) Docker Images
(c) Dockerfile
(d) Docker compose
(e) Volumes
(f) Links

Docker: definiciones

- El **Docker registry/ DockerHub** és un repositorio dónde se cuelgan los ficheros que permiten generar el contenedor (*Docker images*). Una imagen podría ser un fichero que incluye un compilador Java.
- **Imágenes**. Captura del estado de un contenedor. Se utilizan como plantillas para crear contenedores. Existen muchas imágenes publicas (Ubuntu, CentOS) para descargar y que luego adaptamos a nuestra aplicación.
- A partir de las imágenes se generan los **contenedores** (instancias en ejecución de una imagen). A partir de una imagen podemos crear varios contendores.



Docker: definiciones

- **DockerFile**. Es un archivo de texto plano de configuración (similar al *makefile*) que se utiliza para crear imágenes descargada de DockerHub.
- **Docker compose**. Herramienta para definir y ejecutar varios contenedores simultáneamente.
  - Utiliza Dockerfile para configurar la imagen y descargar las dependencias
  - Utiliza Docker-compose.yml para la configuración de los servicios (diferentes contenedores)
  - https://docs.docker.com/compose/gettingstarted
- **Volumes**. Espacios de almacenamiento creados fuera del contenedor. Útil para compartir datos entre contenedores.
- **Link**. Enlazan los contenedores de una máquina entre sí.

Más definiciones en https://docs.docker.com/glossary/

Docker: dockerfile



gcc DOCKER OFFICIAL IMAGE · 10M+ · 738

The GNU Compiler Collection is a compiling system that supports several languages.

```
FROM gcc
RUN addgroup --gid 1000 appgroup
RUN useradd -r --uid 500 -g appgroup appuser
RUN mkdir /home/appuser
RUN chown appuser:appgroup /home/appuser
COPY --chown=appuser:users codi/* /home/appuser/
WORKDIR /home/appuser
USER appuser
```

*An image's **parent image** is the image designated in the FROM directive in the image's Dockerfile. All subsequent commands are based on this parent image. A Dockerfile with the FROM scratch directive uses no parent image, and creates a **base image**.*

https://docs.docker.com/glossary/#parent-image

Docker: ¡Pregunta de examen!

¿Verdadero o Falso? Justifica tu respuesta

Los volúmenes de Docker son espacios de almacenamiento de datos. Al eliminar un contenedor que contiene un volumen montado, los datos del volumen son también eliminados.

Docker: commandos básicos

P8. Lista los comandos básicos de Docker para tratar contenedores e imágenes



Comandos básicos para tratar **contenedores**:

- *ps*: Lista de contenedores ejecutándose
- *ps –a/ ps --all*: Lista de todos los contenedores incluyendo los parados
- *top*: Procesos de un contenedor
- *start*: Levanta un contenedor parado
- *stop*: Para un contenedor en ejecución
- *pause*: Pausa los procesos dentro del contenedor
- *commit*: Crea una imagen desde el contenedor
- *rm*: Eliminar un contenedor parado
- *exec*: Para aplicar un comando dentro de un contenedor ya ejecutándose (docker exec -it myUbuntu /bin/bash)

Docker: commandos básicos

P8. Lista los comandos básicos de Docker para tratar contenedores e imágenes



Comandos básicos para tratar **imágenes**:

- *images*: Lista de imágenes locales
- *run*: Crea un contenedor a partir de una imagen y ejecuta un comando en ella
- *tag*: Etiqueta una imagen
- *pull*: Descarga una imagen del repositorio
- *rmi*: Elimina una imagen local

Docker: puertos

Por defecto, cuando creas un contenedor, **los puertos no son expuestos al exterior**. Para poder hacer que estén disponibles para servicios fuera del Docker o para contenedores fuera de la red del contenedor inicial, es necesario publicarlos mediante el **flag –p**. Esto crea un *firewall* que mapea el puerto del contenedor al puerto del *host* del Docker, permitiendo así la conexión con el exterior.

Flag value	Description
-p 8080:80	Map TCP port 80 in the container to port 8080 on the Docker host.
-p 192.168.1.100:8080:80	Map TCP port 80 in the container to port 8080 on the Docker host for connections to host IP 192.168.1.100.
-p 8080:80/udp	Map UDP port 80 in the container to port 8080 on the Docker host.
-p 8080:80/tcp -p 8080:80/udp	Map TCP port 80 in the container to TCP port 8080 on the Docker host, and map UDP port 80 in the container to UDP port 8080 on the Docker host.

Docker: linkeando contenedores

- Las **redes de tipo *bridge*** son utilizadas por los contenedores ejecutándose **dentro del mismo Docker *host***. Por defecto la propia configuración ya nos proporciona una red local llamada *default* pero es conveniente crear nuestra propia red local para tener una mayor capacidad de configuración.
- En caso de necesidad de conectar con un **Docker *host* exterior**, utilizaremos una red de tipo ***overlay*** o configuraremos el enrutamiento mediante el SO.
- Conectamos el contenedor en ejecución my-Linux a la red local ya creada my-net.
  - docker network connect my-net my-Linux

Docker: kubernetes

- En muchas ocasiones, Docker se utiliza frecuentemente con otras tecnologías como Kubernetes (https://kubernetes.io/).

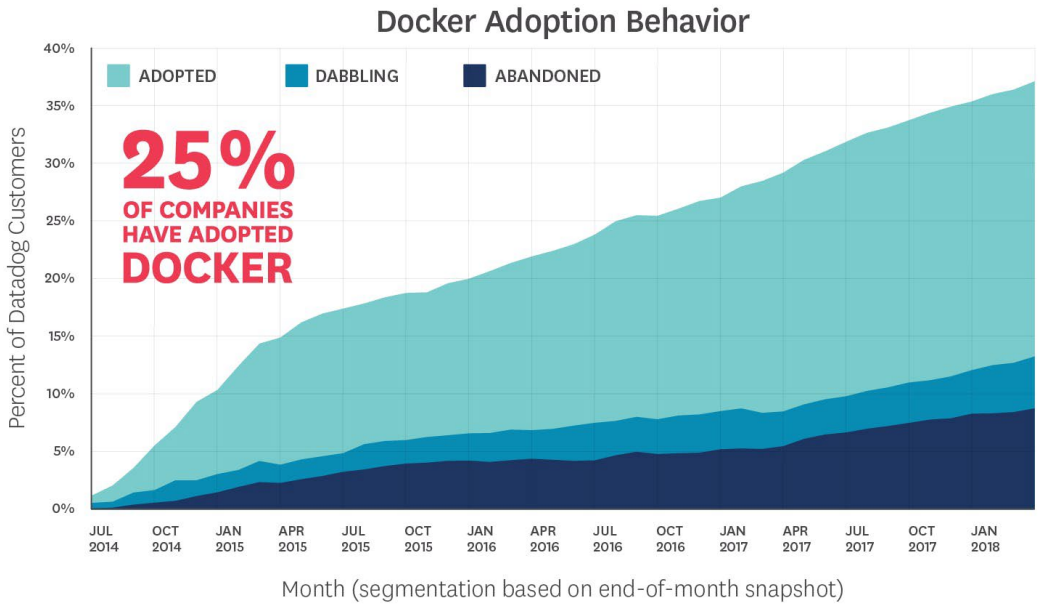


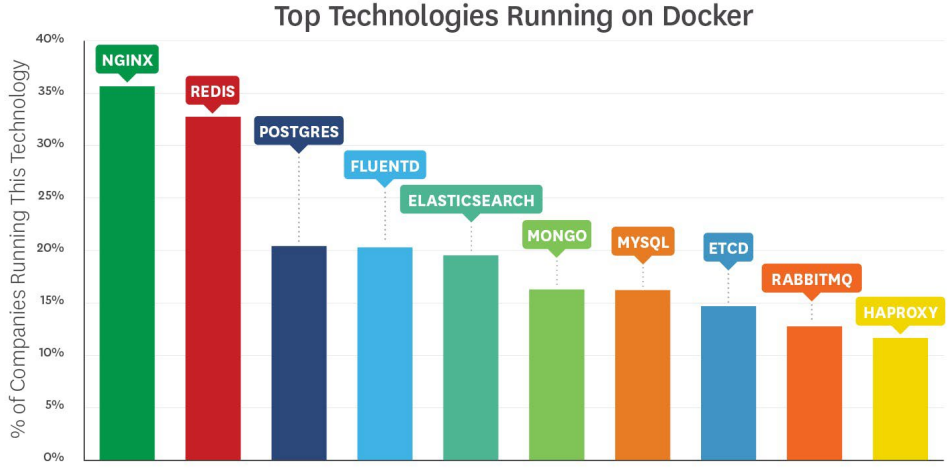
kubernetes

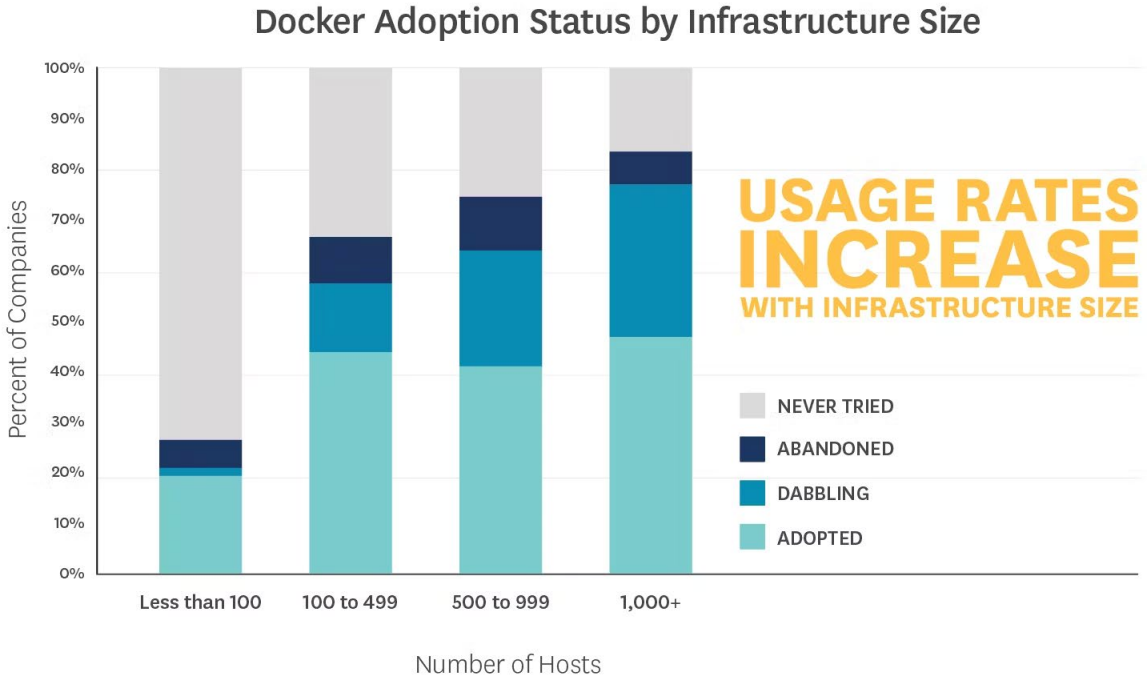
P9. ¿Qué es Kubernetes? ¿ Qué aporta Kubernetes a Docker?



Docker al mundo empresarial (2018)





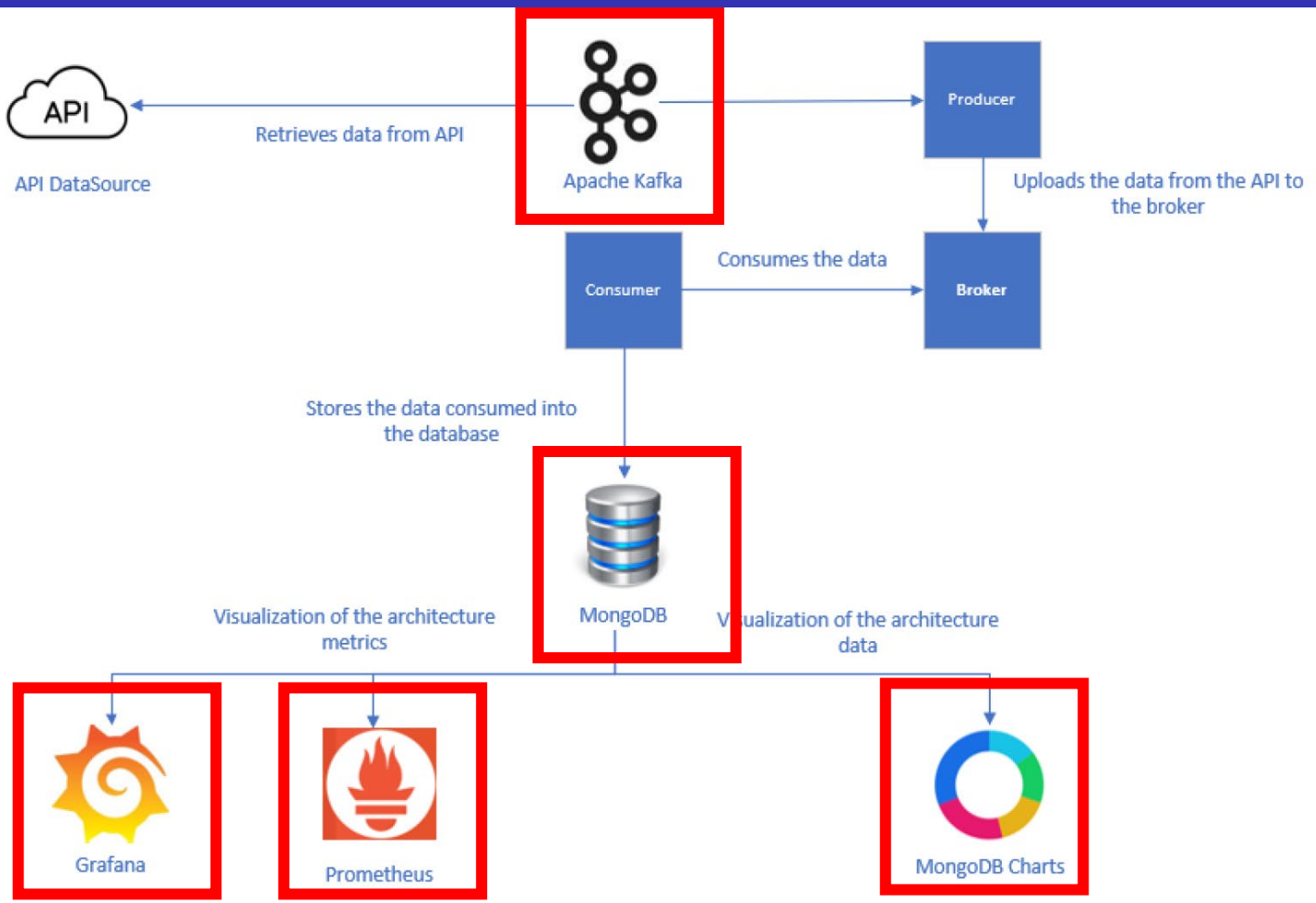


https://www.datadoghq.com/docker-adoption/

Docker al mundo empresarial

- Hay muchas empresas que ofrecen servicios para ejecutar contenedores.
- Al igual que vimos con las MVs, la externalización de este tipo de servicios permiten flexibilidad y escalabilidad.
  - Google Container Engine
  - Amazon EC2 Container Service
  - Azure Container Service
  - Acquia
  - Digital Ocean

Docker al mundo empresarial



Implementacion de una arquitectura end-to-end para la recogida y explotacion de datos desde una API publica. Caso de uso - consumo y visualizacion de datos del COVID-19. Luís Roset. TFG @ UB (2021)

Resumen

- La virtualización permite virtualizar/emular un ordenador.
- En este bloque hemos visto dos formas de conseguirlo:
  - Máquinas virtuales (MV).
  - Contenedores.
- Se puede ejecutar un SO diferente a cada MV. Las MVs permiten limitar los recursos asignados al SO.
- Los contenedores son ejecutados por el SO *host*. A cada contenedor se pueden limitar los recursos asignados. Las llamadas a sistema se realizan directamente al SO.

Otras lecturas recomendadas

- Learning Docker: https://www.linkedin.com/learning/learning-docker-2
- Introduction to containers https://www.coursera.org/learn/ibm-containers-docker-kubernetes-openshift
- Introduction to Dockers: The basics https://www.coursera.org/projects/docker-fundamentals

Frutas (y verduras) de otoño





Test de unidad la semana próxima

